LangChain Section 13: Putting It All Together

Building a Production RAG Chatbot with Agent Capabilities

LangChain v1.2.x (March 2026)

A step-by-step guide to building a realistic, production-ready RAG system with agents, memory, and streaming.

Project: Production RAG Chatbot with Agent Capabilities

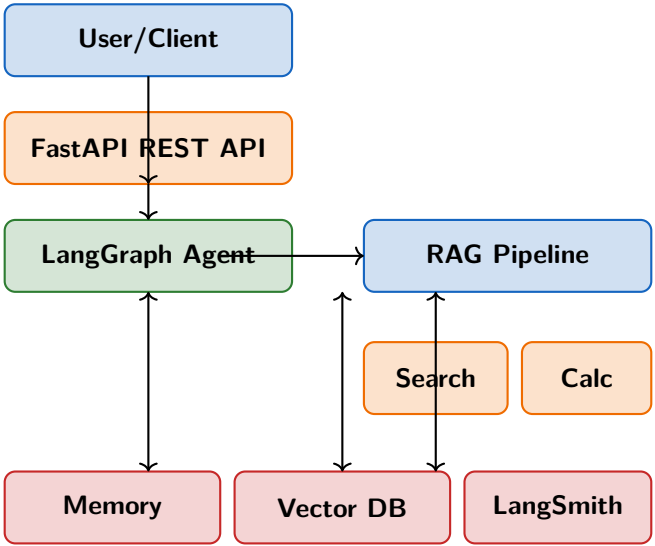
What we're building:

- Intelligent chatbot answering questions about documents
- RAG for contextual, factual responses
- Agent with external tools (web search, calculator)
- Memory across conversation sessions
- Real-time streaming responses
- Full observability with LangSmith
- REST API for easy integration

Technology stack:

- LangChain — orchestration
- LangGraph — agentic workflows
- OpenAI API — LLM (GPT-4)
- Pinecone — vector database
- FastAPI — REST API
- SQLite — memory
- LangSmith — tracing

System Architecture



Project Structure

```
rag_chatbot/
  requirements.txt        -- Dependencies
  .env                    -- API keys
  src/
    ingestion.py         -- Document loading
    retriever.py         -- RAG chain
    tools.py             -- Custom tools
    agent.py             -- LangGraph agent
    memory.py            -- Conversation memory
  api/
    main.py              -- FastAPI app
  run.py                 -- Entry point
```

requirements.txt

```
langchain>=0.1.0
langchain-openai>=0.0.1
langgraph>=0.0.1
langsmith>=0.0.1
fastapi>=0.104.0
uvicorn>=0.24.0
openai>=1.3.0
pinecone-client>=3.0.0
python-dotenv>=1.0.0
```

Step 1: Document Ingestion Pipeline (Part A)

```
from langchain_community.document_loaders
  import PyPDFLoader
from langchain.text_splitter
  import RecursiveCharacterTextSplitter

def load_documents(doc_path):
    loader = PyPDFLoader(doc_path)
    return loader.load()

def split_documents(documents,
    chunk_size=1000):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=200,
        separators=["\n\n", "\n"])
    return splitter.split_documents(documents)
```

Step 1: Document Ingestion Pipeline (Part B)

```
from langchain_openai import OpenAIEmbeddings
from pinecone import Pinecone

def create_vector_store(chunks):
    embeddings = OpenAIEmbeddings(
        model="text-embedding-3-small")

    pc = Pinecone(api_key=os.getenv(
        "PINECONE_API_KEY"))
    index = pc.Index("rag-chatbot-index")

    for chunk in chunks:
        embedding = embeddings.embed_query(
            chunk.page_content)
        index.upsert([(chunk.metadata["source"],
          embedding, chunk.page_content)])

    return index
```

Step 2: Building the Retrieval Chain

```python
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

def create_retrieval_chain(vector_store):
    llm = ChatOpenAI(
        model="gpt-4-turbo-preview",
        temperature=0.7,
        streaming=True)

    rag_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=vector_store.as_retriever(
            k=4))

    return rag_chain
```

Step 3: Adding Tool Capabilities (Part A)

```python
from langchain.tools import Tool
import requests

def search_web(query: str) -> str:
    headers = {"X-API-KEY":
        os.getenv("SERPER_API_KEY")}
    response = requests.get(
        "https://google.serper.dev/search",
        params={"q": query},
        headers=headers)
    results = response.json()
    return str(results.get("organic", []))

def calculate(expression: str) -> str:
    try:
        from sympy import sympify
        return str(sympify(expression))
    except:
        return "Invalid"
```

Step 3: Adding Tool Capabilities (Part B)

```
web_search = Tool(
    name="WebSearch",
    func=search_web,
    description="Search the web")

calc_tool = Tool(
    name="Calculator",
    func=calculate,
    description="Evaluate math")

retrieval_tool = Tool(
    name="DocumentRetrieval",
    func=retrieve_from_docs,
    description="Retrieve from docs")

tools = [web_search, calc_tool,
    retrieval_tool]
```

Step 4: Creating Agent with LangGraph (Part A)

```python
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI

def build_agent_graph(tools):
    def agent_node(state):
        llm = ChatOpenAI(
            model="gpt-4-turbo-preview")
        llm_with_tools = llm.bind_tools(tools)
        messages = state.get("messages", [])
        response = llm_with_tools.invoke(
            messages)
        return {"messages":
            messages + [response]}

    def should_continue(state):
        messages = state.get("messages", [])
        if not messages:
            return END
        return "tools"
```

Step 4: Creating Agent with LangGraph (Part B)

```
    def tool_node(state):
        messages = state.get("messages", [])
        last = messages[-1] if messages else None
        if last and hasattr(last,
            "tool_calls"):
            for call in last.tool_calls:
                name = call.get("name")
                args = call.get("args")
                # Execute tool
        return {"executed": True}

    workflow = StateGraph(dict)
    workflow.add_node("agent", agent_node)
    workflow.add_node("tools", tool_node)
    workflow.add_edge("tools", "agent")
    workflow.add_conditional_edges(
        "agent", should_continue)
    workflow.set_entry_point("agent")
    return workflow.compile()
```

Step 5: Adding Conversation Memory

```python
from langgraph.checkpoint.sqlite
  import SqliteSaver
import sqlite3

def setup_memory(db_path="chat_history.db"):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT
      EXISTS messages (id INTEGER PRIMARY
      KEY, session_id TEXT, role TEXT,
      content TEXT, timestamp DATETIME)''')
    conn.commit()
    return SqliteSaver(connection=conn)

def save_message(session_id, role, content):
    conn = sqlite3.connect("chat_history.db")
    cursor = conn.cursor()
    cursor.execute('''INSERT INTO messages
      (session_id, role, content)
      VALUES (?, ?, ?)''',
      (session_id, role, content))
    conn.commit()
```

Step 6: Adding Streaming Support

```python
from typing import AsyncIterator

async def stream_agent_response(
    executor, user_input, session_id
) -> AsyncIterator[str]:
    history = load_conversation(session_id)
    messages = []
    for m in history:
        if m.get("role") == "user":
            messages.append(
              HumanMessage(m.get("content")))
        else:
            messages.append(
              AIMessage(m.get("content")))

    messages.append(HumanMessage(user_input))
    async for event in
        executor.astream_events(messages):
        if "stream" in event:
            yield event
```

Step 7: Adding LangSmith Tracing

```python
import os

def setup_langsmith():
    os.environ["LANGCHAIN_TRACING_V2"] = "true"
    os.environ["LANGCHAIN_PROJECT"] =
      "rag-chatbot"
    os.environ["LANGCHAIN_ENDPOINT"] =
      "https://api.smith.langchain.com"
    os.environ["LANGCHAIN_API_KEY"] =
      os.getenv("LANGSMITH_API_KEY")

# Every agent run is automatically traced
# Traces include:
#   - Full conversation flow
#   - Tool invocations
#   - Token counts and latency
#   - Errors and exceptions
```

Benefits

Monitor costs, debug flows, A/B test prompts

Step 8: Building the API Layer (Part A)

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="RAG Chatbot")

class ChatMessage(BaseModel):
    session_id: str
    user_input: str

class ChatResponse(BaseModel):
    session_id: str
    response: str

@app.post("/chat")
async def chat(message: ChatMessage):
    history = load_conversation(
        message.session_id)
    result = await executor.ainvoke(history)
    save_message(message.session_id,
      "assistant", result)
    return ChatResponse(
        session_id=message.session_id,
        response=result)
```

Step 8: Building the API Layer (Part B)

```
from fastapi.responses \
  import StreamingResponse

@app.post("/chat/stream")
async def chat_stream(message: ChatMessage):
    async def stream_generator():
        try:
            async for token in \
                stream_agent_response(executor,
                message.user_input,
                message.session_id):
                yield "data: " + token + "\n\n"
        except Exception as e:
            yield "data: ERROR\n\n"

    return StreamingResponse(
        stream_generator(),
        media_type="text/event-stream")

@app.get("/history/{session_id}")
async def get_history(session_id: str):
    return load_conversation(session_id)
```

Step 9: Error Handling and Fallbacks

```python
import logging
from functools import wraps

logger = logging.getLogger(__name__)

def with_fallback(fallback_response):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            try:
                return await func(
                    *args, **kwargs)
            except Exception as e:
                logger.error(str(e))
                return fallback_response
        return wrapper
    return decorator

@with_fallback("Could not process request")
async def agent_call(user_input):
    return await executor.ainvoke(
        {"input": user_input})
```

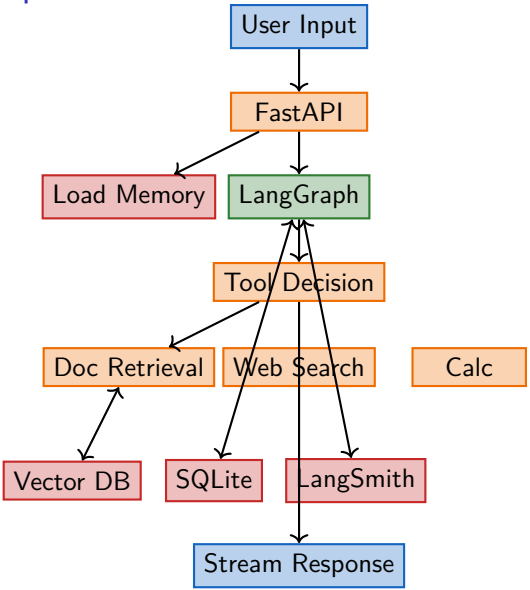
Step 10: Testing the System

```python
import pytest
from langchain.schema import HumanMessage

@pytest.mark.asyncio
async def test_agent_with_tool():
    executor = build_agent_executor()
    msg = HumanMessage(
        content="What is 15 * 23?")
    result = await executor.ainvoke(
        {"messages": [msg]})
    assert "345" in str(result)

@pytest.mark.asyncio
async def test_document_retrieval():
    executor = build_agent_executor()
    msg = HumanMessage(
        content="What does section 3 say?")
    result = await executor.ainvoke(
        {"messages": [msg]})
    assert len(result) > 0
```

Complete Data Flow



Entry Point: run.py

```python
import asyncio
from dotenv import load_dotenv

async def main():
    load_dotenv()
    setup_langsmith()
    setup_memory()

    print("Indexing documents...")
    vector_store = ingest_all(
        "./data/documents")

    print("Building agent...")
    agent = build_agent_executor(
        vector_store)

    print("Starting API...")
    import uvicorn
    uvicorn.run("api.main:app",
      host="0.0.0.0", port=8000)

if __name__ == "__main__":
    asyncio.run(main())
```

Running the Application

Step 1: Setup

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Step 2: Configure

```
OPENAI_API_KEY=sk-...
PINECONE_API_KEY=...
LANGSMITH_API_KEY=...
```

Step 3: Run

```
python run.py
```

Testing the API

Using curl

```
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"session_id":"user123",
       "user_input":"What is section 3?"}'
```

Using Python

```
import requests
response = requests.post(
    "http://localhost:8000/chat",
    json={"session_id": "user123",
          "user_input": "Calculate 42*17"})
print(response.json())
```

Recap: What We Used From Each Section

Task	Component	Section
Load PDFs	PyPDFLoader	1
Split text	RecursiveCharacterTextSplitter	1
Embeddings	OpenAIEmbeddings	3
Vector search	Pinecone	3
Chains	RetrievalQA	2
Memory	SqliteSaver	4
Tools	Tool class	5
Agent	StateGraph	7
Streaming	async iterators	8
API	FastAPI	9
Tracing	LangSmith	10

Key Lessons: Building Production Systems

- **Start simple:** Begin with basic RAG, add agents later
- **Memory matters:** Persistent history is essential
- **Streaming improves UX:** Real-time feedback matters
- **Observability is critical:** LangSmith reveals bottlenecks
- **Tool selection is hard:** Agents need clear descriptions
- **Error handling required:** Plan for API/retrieval failures
- **Testing essential:** Mock tools for fast iteration
- **Monitor costs:** Track token usage carefully
- **Security first:** Never commit API keys

Enhancement Ideas

Capability

- Multi-turn reasoning
- Document citations
- Clarification questions
- Follow-up suggestions

Integration

- Slack/Teams bot
- Webhooks
- Database integration
- File upload

System

- Prompt optimization
- Fine-tuning on Q&A
- Rate limiting
- Web UI

Monitoring

- A/B testing
- User satisfaction
- Cost tracking
- Alerts

Advanced: Multi-Agent Collaboration

```
supervisor_prompt = "You supervise agents"

def supervisor_node(state):
    messages = state.get("messages", [])
    llm = ChatOpenAI(
        model="gpt-4-turbo-preview")
    route = llm.invoke(messages)
    return {"next_agent": route.next}

workflow = StateGraph(dict)
workflow.add_node("supervisor",
    supervisor_node)
workflow.add_node("retrieval_agent",
    retrieval_agent)
workflow.add_node("web_agent", web_agent)
workflow.add_conditional_edges(
    "supervisor",
    lambda x: x.get("next_agent", "end"))
```

Advanced: Custom Memory Types

```python
class ContextualMemory:
    def __init__(self, max_tokens=2000):
        self.messages = []
        self.max_tokens = max_tokens

    def add(self, message):
        self.messages.append(message)
        self._prune_old_messages()

    def _prune_old_messages(self):
        embeddings = OpenAIEmbeddings()
        # Score by relevance
        # Keep high-scoring

    def get_relevant(self, query):
        # Return relevant messages
        pass
```

Advanced: Dynamic Tool Selection

```
def select_best_tools(query):
    embeddings = OpenAIEmbeddings()
    query_emb = embeddings.embed_query(query)

    tool_scores = []
    for tool in all_tools:
        desc_emb = embeddings.embed_query(
            tool.description)
        score = cosine_similarity(
            query_emb, desc_emb)
        tool_scores.append((tool, score))

    return [t for t, s in tool_scores
            if s > 0.7]

def smart_agent(state):
    messages = state.get("messages", [])
    query = messages[-1].content if messages
    tools = select_best_tools(query)
    return ChatOpenAI().bind_tools(
        tools).invoke(messages)
```

Deployment Patterns

Development

- Local SQLite
- Mock LLM
- Fast iteration

Production

- PostgreSQL
- Real API
- Rate limiting

Docker

- Container-based
- Kubernetes
- Isolation

Serverless

- AWS Lambda
- Auto-scaling
- Pay-per-call

Docker Deployment

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
  -r requirements.txt
COPY . .
ENV PYTHONUNBUFFERED=1
CMD ["python", "run.py"]
```

```
version: '3'
services:
  chatbot:
    build: .
    ports:
      - "8000:8000"
    environment:
      OPENAI_API_KEY: APIkey
      PINECONE_API_KEY: APIkey
```

Monitoring and Logging

```python
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(message)s",
    handlers=[
        logging.FileHandler("chatbot.log"),
        logging.StreamHandler()
    ])

logger = logging.getLogger(__name__)

def agent_node(state):
    msgs = state.get("messages", [])
    logger.info(f"Processing {len(msgs)} msgs")
    try:
        response = llm.invoke(msgs)
        logger.info(f"Response: {len(response)}")
        return response
    except Exception as e:
        logger.error(f"Error: {e}")
        raise
```

Cost Optimization Strategies

- Use smaller models for simple tasks
- Batch requests to amortize API calls
- Cache embeddings and pre-compute
- Limit retriever scope with ranking
- Monitor token usage per conversation
- Use function calling efficiently
- Implement request quotas
- A/B test prompts for cost/quality

Cost Tracking

Log token usage per request and aggregate in LangSmith.

Security Best Practices

- **Environment variables:** Never hardcode keys
- **Input validation:** Sanitize user queries
- **Rate limiting:** Prevent abuse
- **Authentication:** Require API tokens
- **HTTPS only:** Encrypt in transit
- **Data retention:** Delete old conversations
- **Audit logs:** Track access
- **Isolation:** Containers with minimal privileges

Performance Tuning

Latency

- Async/await for I/O
- Parallel tool execution
- Cache frequent queries
- Connection pooling

Throughput

- Load balancing
- Worker processes
- Horizontal scaling
- Queue-based architecture

Profiling

Use cProfile and LangSmith traces to find bottlenecks.

Debugging Common Issues

Issue	Solution
Agent won't use tools	Check descriptions
Poor retrieval	Tune chunk size
Tool errors	Add error handling
Memory bloat	Implement pruning
Slow responses	Profile with LangSmith
Inconsistent outputs	Tune temperature
Token limit exceeded	Summarize

Self-Check Questions (1/2)

1. What are the three RAG components and why each matters?
2. How does StateGraph differ from a simple chain?
3. What is checkpointing in memory?
4. How to implement streaming in FastAPI?

Self-Check Questions (2/2)

5. Why is dynamic tool selection via embeddings better?
6. Three ways to optimize cost in production RAG?
7. Debug an agent that never uses web search?
8. What to add for enterprise use?

What's Next? Advanced Topics

- **Fine-tuning:** Adapt to your domain
- **Multi-modal RAG:** Images, videos, tables
- **Graph RAG:** Knowledge graphs
- **Recursive summarization:** Large documents
- **Adversarial robustness:** Handle jailbreaks
- **Custom evaluation:** Correctness checks
- **Federated learning:** Distributed data
- **Real-time updates:** Zero-downtime deployment

Summary: Key Takeaways

The Complete System

A production RAG chatbot requires:

- Ingestion: Load, split, embed documents
- Retrieval: Index and retrieve vectors
- Agents: Select and use tools
- Memory: Persist context
- APIs: Expose via REST with streaming
- Observability: Trace every call
- Production: Deploy, scale, monitor

Remember

Start simple (basic RAG), validate, then add complexity. Every feature has a cost.

Resources and References

- **LangChain:** https://python.langchain.com
- **LangGraph:** https://langchain-ai.github.io/langgraph
- **LangSmith:** https://docs.smith.langchain.com
- **OpenAI API:** https://platform.openai.com/docs
- **Pinecone:** https://docs.pinecone.io
- **FastAPI:** https://fastapi.tiangolo.com
- **Examples:** LangChain GitHub

You Now Know...

How to build a production-grade LangChain application from scratch

- Document ingestion at scale
- Retrieval-augmented generation
- Multi-tool agent orchestration
- Persistent memory
- Real-time streaming
- Full observability
- Deployment and monitoring

The next step: Build your own and iterate!

Thank you!

Questions?

Let's discuss implementation, architecture, or your project ideas.

[Docs: LangChain v1.2.x Documentation]